Natural Language Processing

Autumn 2026

Week 2: Word Embeddings

Workshop starts this week

- It is important for you to come to the workshops. It gives you a better learning experience and helps you to form groups for Assignment 3.

- A/Prof Wei Liu <Wei.Liu@uts.edu.au>
  Wednesday workshop, 6 pm to 7 pm.

Contact Wei for questions related to assessments, deadline extension, and any general questions.

- Miss Bonny (Li-Chiao) Wang <Bonny.Wang@uts.edu.au>
  Wednesday workshops, 3 pm to 6 pm.

- Mr Jin Li <Jin.Li@uts.edu.au>
  Wednesday 7 pm to 9 pm and Thursday workshops.

Workshops

- You may occasionally attend a workshop session other than your allocated one, except in Weeks 6 and 10.
- In Weeks 6 and 10, you must attend the workshop shown in your timetable to complete the Quiz (Assignment 2).
- For Assignment 3, you may form groups with students from other workshop sessions.
  - However, your group must enter member details into the workshop session where you intend to present in Week 12.
  - For example, if your group contain members from two different workshops, you will need to choose one of these two workshops and entre all member details there.
  - Instructions for submitting group membership are provided in the Assignment 3 description on Canvas.

Week 2: Table of Contents

Word Embeddings

- Introduction
- One-Hot vectors
- Bag of Words
- Word2Vec
- GloVe

Word Representation

- Computers do not understand words. They can only understand numbers.
- In natural language processing (NLP), representing words in a form that computational models can interpret is crucial.
- Representing words in a numerical format effectively offers significant benefits in NLP:
  - Semantic understanding (capturing the meanings and relationships between words)
  - Reducing dimensionality (representing words in low-dimensional vectors)
  - Contextual flexibility (distinguishing between different meanings of the same word based on context)

Representing words as discrete symbols

In traditional NLP, we regard words as discrete symbols:
hotel, conference, motel – a localist representation

Represented by 1, the rest 0s

Words can be represented by vectors:

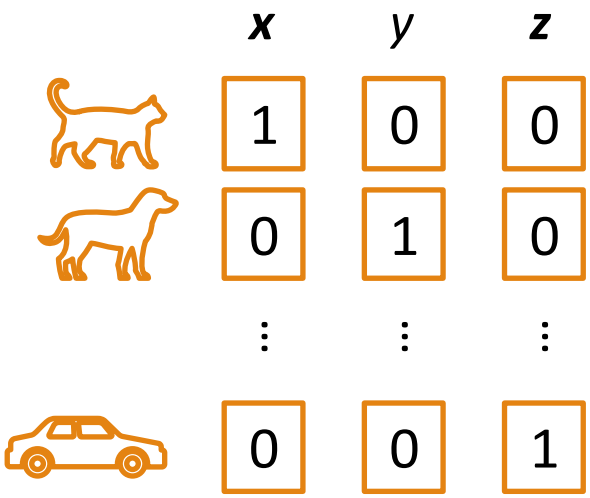
motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

Problem: Vector dimension = number of words in vocabulary (e.g., 500,000)

One-hot vectors

One-hot vectors map objects/words into fixed-length vectors

These vectors only contain the identity information, not semantic meaning.



Problem with words as discrete symbols

Example: in web search, if user searches for "Sydney motel", we would also like to match documents containing "Sydney hotel". But if using one-hot vectors:

Word	Vector
hotel	001000
motel	000100
coffee	000001

Problem: Distance(hotel, motel) = Distance(hotel, coffee). There is no natural notion of **similarity** for one-hot vectors!

Solution: Learn to encode similarity in the vectors themselves

Bag of Words (BoW)

- A simple text representation technique in NLP.
- Represents text as a collection (or "bag") of individual words.
- Ignores grammar and word order but preserves frequency.

Image credit: dataaspirant.com

How Does Bag of Words Work?

1. Tokenisation: Splitting text into words.
2. Creating a Vocabulary: Listing unique words from all texts.
3. Encoding Text: Converting sentences into word frequency vectors.

Image credit: https://tinyurl.com/3mfjsar7

Bag of Words - Examples

1. **"The cat sat on the mat. The cat is happy."**

2. **"The dog sat on the mat. The dog is playful."**

Vocabulary: [The, cat, sat, on, mat, is, happy, dog, playful]

BoW Representation:

Sentence	The	cat	sat	on	mat	is	happy	dog	playful
"The cat sat on the mat. The cat is happy."	2	2	1	1	1	1	1	0	0
"The dog sat on the mat. The dog is playful."	2	0	1	1	1	1	0	2	1

- The word "The" appears twice in both sentences, so it gets a count of 2 in both vectors.
- The word "cat" appears twice in the first sentence but not at all in the second.
- The word "dog" appears twice in the second sentence but not at all in the first.
- Other words like "sat," "on," "mat," and "is" appear only once per sentence.

You may have noticed how BoW captures word frequency but not word order or meaning.

What do you think are the advantages and limitations of BoWs?

Advantages and Limitations of Bag of Words

- Advantages:
  - Simple and easy to implement.
  - Efficient for small datasets.
  - Works well with basic machine learning models.

- Limitations:
  - Ignores word order and context.
  - High-dimensional for large vocabularies.
  - Does not capture **semantic** meaning (e.g., synonyms are treated differently).

Preliminary for Word2Vec: Dense vector

A dense vector refers to a vector where most of the elements are non-zero.

Vector Type	Example Representation for "Cat"
Dense Vector	`[0.12, -0.45, 0.78, ..., -0.34]` (e.g., 300 dimensions)
Sparse Vector	`[0, 0, 0, 1, 0, 0, ..., 0]` (e.g., 10,000+ dimensions)

Why dense vector?

- Dense vectors capture complex relationships between words, encoding semantic meanings and contexts.
- Dense vectors have lower dimensionality compared to sparse vectors, leading to reduced memory usage and faster computations.
- In real-world applications, people have found that models utilising dense vectors often outperform those using sparse representations.

Word vectors

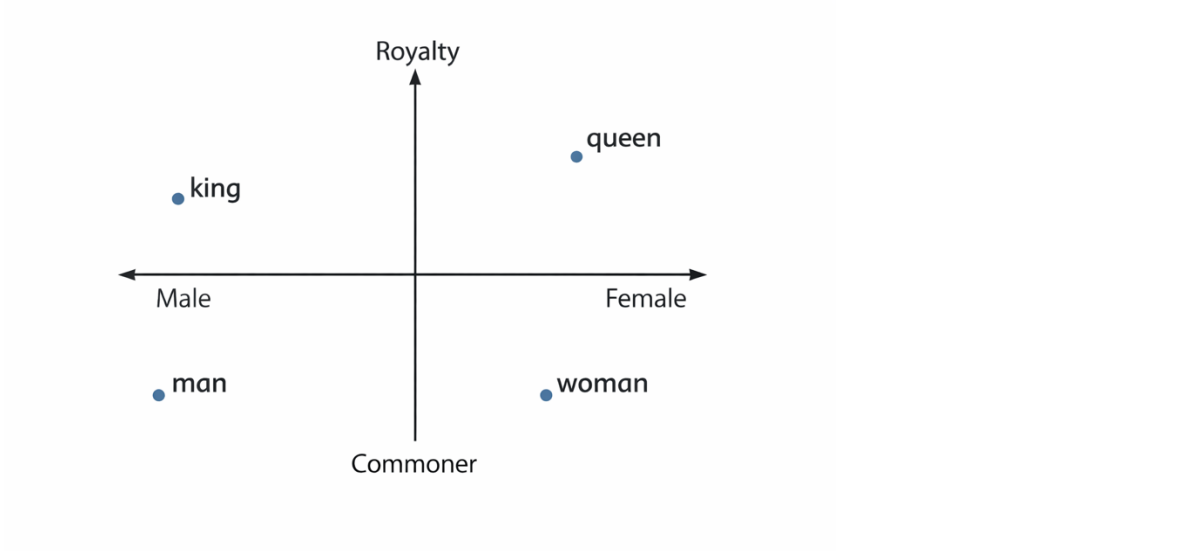
We will build a **dense vector** for each word, chosen so that it is similar to vectors of words that appear in similar contexts

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Note: word vectors are also called word embeddings or word representations.

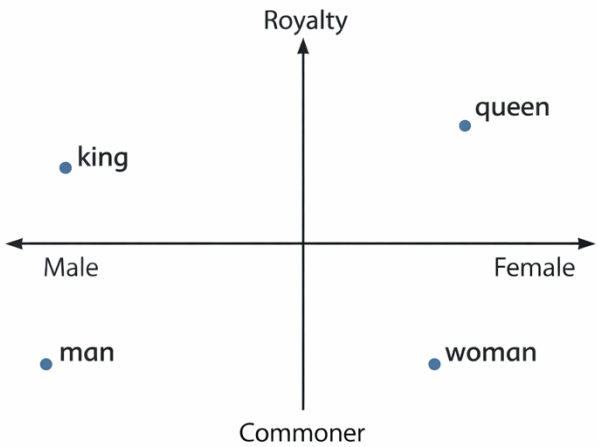
Word meaning as a word vector – visualization

- Think of word embeddings for four words, “king”, “queen”, “man” and “woman”, as coordinates in a 2D space:



- Observation: *king – man + woman ≈ queen*

Why is this useful?



- By using the same approach, we can imagine:
  - *Paris – France + Italy ≈ Rome*
  - *walk – walking + swimming ≈ swim*

- We are no longer matching words. We are essentially doing math on meaning.

How can we capture a word's semantic meaning?

- How did you learn new English words when you were in primary school?
- Did your English teacher suggest you look up words in a dictionary every time you meet a new word?
- Or did your teacher suggest you look at the "**context**" of the new word first?
- How can we build in "context" to better capture semantics?

Context Matters

- Which of the following words is the closest to “apple”?

A. banana
B. microsoft
C. chair

Representing words by their context

- Distributional semantics: **A word's meaning is given by the words that frequently appear close-by**
  - *"You shall know a word by the company it keeps"* (J. R. Firth 1957: 11)
  - One of the most successful ideas of modern statistical NLP
- When a word *w* appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- Use the many contexts of *w* to build up a representation of *w*

…government debt problems turning into	***banking***	*crises as happened in 2009…*
…saying that Europe needs unified	***banking***	*regulation to replace the hodgepodge…*
…India has just given its	***banking***	*system a shot in the arm…*

These context words will represent ***banking***

Word2vec: Overview

Word2vec (Mikolov et al. 2013) is a framework for learning word vectors

Idea:

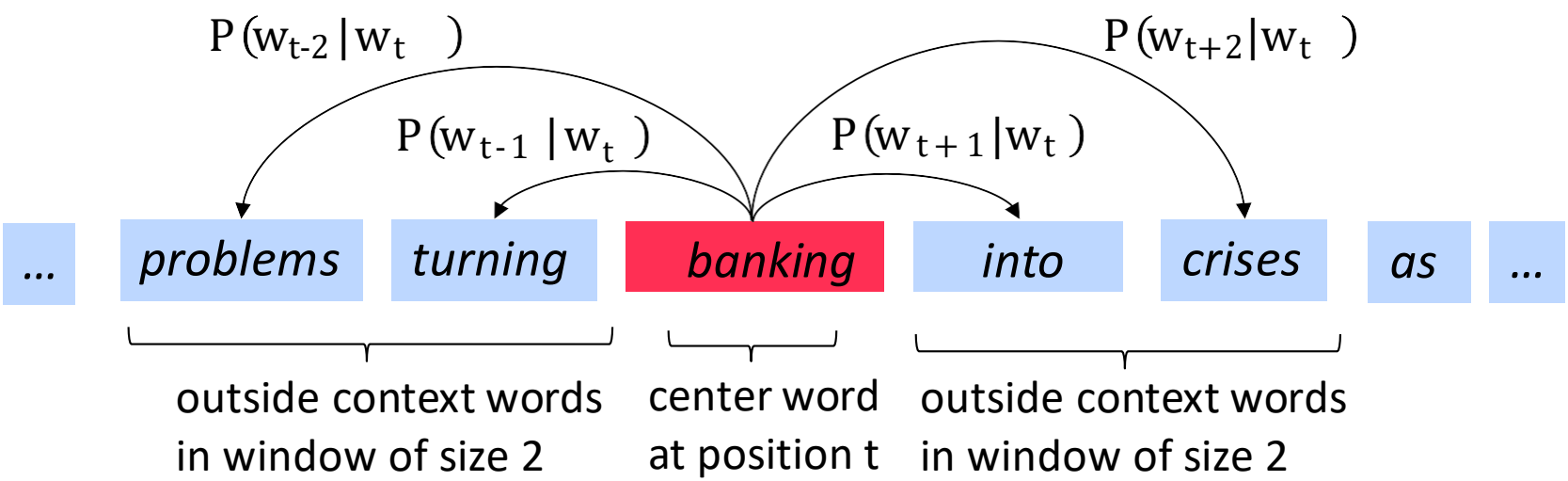
- We have a large corpus of text
- Every word in a fixed vocabulary is represented by a vector
- Go through each position *t* in the text, which has a center word *c* and context ("outside") words *o*
- Use the similarity of the word vectors for *c given* o (continuous bag-of-words, or CBOW) and *o* to calculate the probability of *o* given *c* (skip-gram)
- Keep adjusting the word vectors to maximize this probability

Word2Vec Overview

- We will use two vectors per word: v_w when w is the center word and u_w when w is the context word

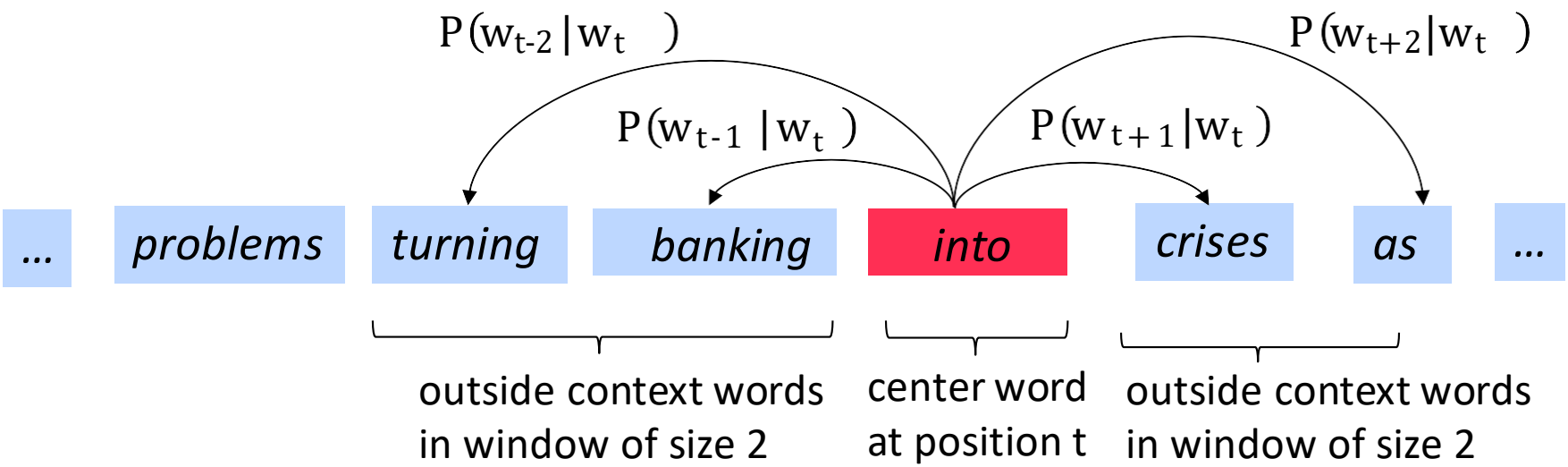
$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

- Example windows and process for computing $P(w_{t+j} | w_t)$



Word2Vec Overview

- Example windows and process for computing $P(w_{t+j} \mid w_t)$



Word2vec: objective function

For each position $t = 1, \ldots, T$, predict context words within a window of fixed size m, given center word w_j

Likelihood = $$L(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m} P(w_{t+j} \mid w_t; \theta)$$

θ is all variables to be optimized

sometimes called *cost* or *loss* function

The objective function J is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{-m \le j \le m} \log P(w_{t+j} \mid w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

What do you think the limitation of Word2Vec would be?

Limitation of Word2Vec

Word2Vec might miss broader corpus-wide relationships because it focuses only on nearby words.

- Imagine we have a small training corpus (*training corpus* is the collection of text a model learns language from) with the following sentences:
  1. "The cat sat on the mat."
  2. "The kitten is playing on the floor."
  3. "The dog sleeps in the kennel."
  4. "A lion is a wild animal."

- Word2Vec learns that "cat" frequently appears near "sat", "on", and "mat."
- However, it struggles to capture the semantic relationship between "cat" and other related words like "kitten" or "lion" unless they happen to appear within the same small context window. This results in a limited understanding of global word relationships.

GloVe (Global Vectors for Word Representation)

- GloVe (Global Vectors for Word Representation) captures *global* word co-occurrence.
- The Core Idea: Co-occurrence Matrix. A co-occurrence matrix counts how often words appear together in a defined window, where
  - Rows represent words (target words).
  - Columns represent words that appear in the context of the target word.
  - Values at location (i,j) represent how often word j appears in the context of word i.
- Example - Suppose we have a tiny corpus with two sentences: "*The cat sat. The dog ran*." With a window size of 4, the co-occurrence matrix might look like:

GloVe use this matrix to create word embeddings.

context	the	cat	sat	dog	ran
the	1	2	2	2	1
cat	2	0	1	1	1
sat	2	1	0	1	1
dog	2	1	1	0	1
ran	1	1	1	1	0

Comparisons of Word Representation Methods

1. **One-hot Vector:** Represents words as binary vectors where only one dimension is '1' and all others are '0', indicating the presence of a word in a vocabulary.
2. **Bag of Words (BoW):** Represents text as word frequency counts without considering word order or meaning.
3. **Word2Vec:** Learn dense word embeddings based on local context in sentences.
4. **GloVe:** Learn word representations by capturing global context.

While Word2Vec and GloVe are expected to outperform one-hot vectors and BoW, which is better between Word2Vec and GloVe depends on the scenarios (whether local or global information is more important in a scenario).

What if there are unknown words?

The Rare Word Problem

- Example: "*I was surprised by the unbelievableness of the story.*"
- Word2Vec vocabulary:

  believable ✓
  unbelievable ✓
  unbelievableness ✗ *(unknown word)*

- In this case, Word2Vec will ignore the word, or produce a special token <UNK>, representing "unknown", which is not ideal.

Represent words using subwords

Subword Embeddings

- Word = sum of subword vectors
- Example:

  unbelievableness =
  <unbel> + <nbeli> + <belie> + <eliev> + <lieva> + <ievab> +
  <evabl> + <vable> + <ablen> + <blene> + <lenes> + <eness>

- Consequence: Unseen words get meaningful vectors using previously seen words' subwords.
- Even if the model never saw the whole word "unbelievableness", it has seen most of its pieces before from words like "unbelievable" and "kindness", so it can infer its meaning.

Questions?